MIPS Reference Data

A More Readable Edition

Reformatted from the Patterson & Hennessy MIPS "Green Card"
Computer Organization and Design, 4th ed., © 2009 Elsevier

Contents

1 How to Use This Sheet **2**

2 Instruction Formats **2**

3 Core Integer Instruction Set **2**
3.1 Arithmetic . . . 3
3.2 Logical & Shift . . . 3
3.3 Data Transfer (Load / Store) . . . 3
3.4 Comparison . . . 4
3.5 Branch & Jump . . . 4
3.6 Field-encoding notes (from the original card) . . . 4

4 Floating-Point Instruction Set **4**
4.1 Arithmetic . . . 4
4.2 Compare & Branch . . . 5
4.3 Data Transfer & Misc. . . . 5

5 Pseudoinstructions **5**

6 Registers **5**
6.1 Register name, number, use, call convention . . . 5
6.2 Memory map and stack frame . . . 6

7 Data Sizes, Alignment & Number Formats **6**
7.1 Data alignment . . . 6
7.2 IEEE 754 floating point . . . 6

8 Exception Handling **7**
8.1 Cause and Status registers . . . 7
8.2 Exception codes . . . 7

9 Opcode & Funct Field Quick Lookup **7**
9.1 Base conversion notes . . . 7

10 Size Prefixes **7**

1 How to Use This Sheet

This is the same reference data as the original MIPS "green card," but reorganized and enlarged for readability:

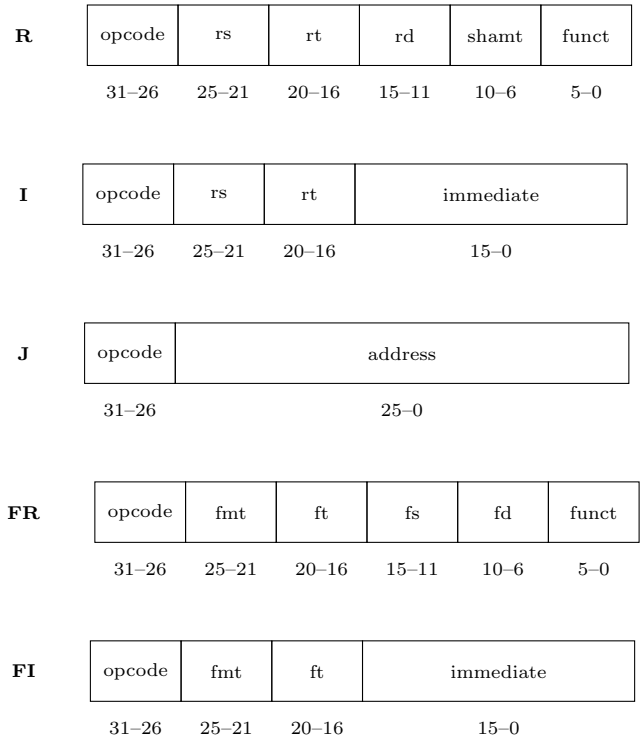
- Instructions are grouped **by category** (arithmetic, logical, data transfer, branch/jump, comparison, floating point) instead of strict alphabetical order.
- Every table uses a font large enough to read without a magnifying glass.
- Field/bit diagrams (instruction formats, stack frame, memory map, exception registers) are redrawn as clean boxes.

Common notation used in the "Operation" column:

Notation	Meaning
`R[rs]`, `R[rt]`, `R[rd]`	value in the named general-purpose register
`M[addr]`	value in memory at address `addr`
`F[fs]`, `F[ft]`, `F[fd]`	value in the named floating-point register
`SignExtImm`	16-bit immediate, sign-extended to 32 bits
`ZeroExtImm`	16-bit immediate, zero-extended to 32 bits
`PC`	program counter
`Hi`, `Lo`	the two special multiply/divide result registers

2 Instruction Formats

Every MIPS instruction is 32 bits, in one of five layouts:



3 Core Integer Instruction Set

Format legend: **R** = register, **I** = immediate, **J** = jump. The last column gives `opcode/funct` in hex ("`0/20`" means opcode `0` with `funct` field `0x20`; a bare hex value is the opcode alone, used when there is no funct field).

3.1 Arithmetic

Name	Mnemonic	Fmt	Operation	Opcode/Funct
Add	`add`	R	`R[rd] = R[rs] + R[rt]`	0/20
Add Immediate	`addi`	I	`R[rt] = R[rs] + SignExtImm`	8
Add Unsigned	`addu`	R	`R[rd] = R[rs] + R[rt]`	0/21
Add Imm. Unsigned	`addiu`	I	`R[rt] = R[rs] + SignExtImm`	9
Subtract	`sub`	R	`R[rd] = R[rs] -- R[rt]`	0/22
Subtract Unsigned	`subu`	R	`R[rd] = R[rs] -- R[rt]`	0/23
Multiply	`mult`	R	`{Hi,Lo} = R[rs] * R[rt]`	0/18
Multiply Unsigned	`multu`	R	`{Hi,Lo} = R[rs] * R[rt]`	0/19
Divide	`div`	R	`Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]`	0/1a
Divide Unsigned	`divu`	R	`Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]`	0/1b
Move From Hi	`mfhi`	R	`R[rd] = Hi`	0/10
Move From Lo	`mflo`	R	`R[rd] = Lo`	0/12

Add, Add Immediate and Subtract may cause an overflow exception; their Unsigned counterparts never do.

3.2 Logical & Shift

Name	Mnemonic	Fmt	Operation	Opcode/Funct
And	`and`	R	`R[rd] = R[rs] & R[rt]`	0/24
And Immediate	`andi`	I	`R[rt] = R[rs] & ZeroExtImm`	c
Or	`or`	R	`R[rd] = R[rs] | R[rt]`	0/25
Or Immediate	`ori`	I	`R[rt] = R[rs] | ZeroExtImm`	d
Nor	`nor`	R	`R[rd] = ~(R[rs] | R[rt])`	0/27
Shift Left Logical	`sll`	R	`R[rd] = R[rt] <<shamt`	0/00
Shift Right Logical	`srl`	R	`R[rd] = R[rt] >>shamt`	0/02
Shift Right Arithmetic	`sra`	R	`R[rd] = R[rt] >>>shamt`	0/03

3.3 Data Transfer (Load / Store)

Name	Mnemonic	Fmt	Operation	Opcode
Load Word	`lw`	I	`R[rt] = M[R[rs]+SignExtImm]`	23
Store Word	`sw`	I	`M[R[rs]+SignExtImm] = R[rt]`	2b
Load Byte Unsigned	`lbu`	I	`R[rt] = {24'b0, M[R[rs]+SignExtImm](7:0)}`	24
Store Byte	`sb`	I	`M[R[rs]+SignExtImm](7:0) = R[rt](7:0)`	28
Load Halfword Unsigned	`lhu`	I	`R[rt] = {16'b0, M[R[rs]+SignExtImm](15:0)}`	25
Store Halfword	`sh`	I	`M[R[rs]+SignExtImm](15:0) = R[rt](15:0)`	29
Load Upper Immediate	`lui`	I	`R[rt] = {immediate, 16'b0}`	f
Load Linked	`ll`	I	`R[rt] = M[R[rs]+SignExtImm]`	30
Store Conditional	`sc`	I	`M[..]=R[rt]; R[rt] = (atomic)?1:0`	38

`ll`/`sc` form an atomic test-and-set pair: `sc` stores its value and returns 1 in `R[rt]` only if the memory word has not been touched since the matching `ll`; otherwise it fails and returns 0.

3.4 Comparison

Name	Mnemonic	Fmt	Operation	Opcode/Funct
Set Less Than	`slt`	R	`R[rd] = (R[rs]<R[rt]) ? 1 : 0`	`0/2a`
Set Less Than Immediate	`slti`	I	`R[rt] = (R[rs]<SignExtImm) ? 1:0`	`a`
Set Less Than Unsigned	`sltu`	R	`R[rd] = (R[rs]<R[rt]) ? 1:0` (unsigned)	`0/2b`
Set Less Than Imm. Unsigned	`sltiu`	I	`R[rt] = (R[rs]<SignExtImm) ? 1:0` (unsigned)	`b`

3.5 Branch & Jump

Name	Mnemonic	Fmt	Operation	Opcode
Branch On Equal	`beq`	I	`if(R[rs]==R[rt]) PC=PC+4+BranchAddr`	`4`
Branch On Not Equal	`bne`	I	`if(R[rs]!=R[rt]) PC=PC+4+BranchAddr`	`5`
Jump	`j`	J	`PC = JumpAddr`	`2`
Jump And Link	`jal`	J	`R[31]=PC+8; PC=JumpAddr`	`3`
Jump Register	`jr`	R	`PC = R[rs]`	`0/08`

3.6 Field-encoding notes (from the original card)

(1) May cause overflow exception.

(2) `SignExtImm = { 16{immediate[15]}, immediate }`

(3) `ZeroExtImm = { 16{1b'0}, immediate }`

(4) `BranchAddr = { 14{immediate[15]}, immediate, 2'b0 }`

(5) `JumpAddr = { PC+4[31:28], address, 2'b0 }`

(6) Operands considered unsigned numbers (vs. 2's complement).

(7) Atomic test&set pair; `R[rt] = 1` if pair atomic, 0 if not atomic.

4 Floating-Point Instruction Set

Formats **FR**/**FI** mirror R/I but with floating-point register fields `fmt` (single `s`=10_{16} / double `d`=11_{16}), `ft`, `fs`, `fd`. Opcode/Funct column shows `opcode/fmt/funct`.

4.1 Arithmetic

Name	Mnemonic	Fmt	Operation	Opcode/Fmt/Funct
FP Add Single	`add.s`	FR	`F[fd] = F[fs] + F[ft]`	`11/10/0`
FP Add Double	`add.d`	FR	`{F[fd],F[fd+1]} = {F[fs],F[fs+1]}+{F[ft],F[ft+1]}`	`11/11/0`
FP Subtract Single	`sub.s`	FR	`F[fd] = F[fs] -- F[ft]`	`11/10/1`
FP Subtract Double	`sub.d`	FR	`{F[fd],F[fd+1]} = {F[fs],F[fs+1]}-{F[ft],F[ft+1]}`	`11/11/1`
FP Multiply Single	`mul.s`	FR	`F[fd] = F[fs] * F[ft]`	`11/10/2`
FP Multiply Double	`mul.d`	FR	`{F[fd],F[fd+1]} = {F[fs],F[fs+1]}*{F[ft],F[ft+1]}`	`11/11/2`
FP Divide Single	`div.s`	FR	`F[fd] = F[fs] / F[ft]`	`11/10/3`
FP Divide Double	`div.d`	FR	`{F[fd],F[fd+1]} = {F[fs],F[fs+1]}/{F[ft],F[ft+1]}`	`11/11/3`

4.2 Compare & Branch

Name	Mnemonic	Fmt	Operation	Opcode/Fmt/Funct
FP Compare Single	`c.`*x*`.s`*	FR	`FPcond = (F[fs] op F[ft]) ? 1:0`	`11/10/`*y*
FP Compare Double	`c.`*x*`.d`*	FR	`FPcond = ({F[fs],F[fs+1]} op {F[ft],F[ft+1]}) ? 1:0`	`11/11/`*y*
Branch On FP True	`bc1t`	FI	`if(FPcond) PC=PC+4+BranchAddr`	`11/8/1`
Branch On FP False	`bc1f`	FI	`if(!FPcond) PC=PC+4+BranchAddr`	`11/8/0`

* *x is* `eq`*,* `lt`*, or* `le`*; op is* `==`*,* <*, or* `<=`*; y is* `32`*,* `3c`*, or* `3e` *respectively.*

4.3 Data Transfer & Misc.

Name	Mnemonic	Fmt	Operation	Opcode
Load FP Single	`lwc1`	I	`F[rt] = M[R[rs]+SignExtImm]`	`31`
Store FP Single	`swc1`	I	`M[R[rs]+SignExtImm] = F[rt]`	`39`
Load FP Double	`ldc1`	I	`F[rt]=M[..]; F[rt+1]=M[..+4]`	`35`
Store FP Double	`sdc1`	I	`M[..]=F[rt]; M[..+4]=F[rt+1]`	`3d`
Move From Control	`mfc0`	R	`R[rd] = CR[rs]`	`10/0/0`

5 Pseudoinstructions

Convenience instructions the assembler expands into one or more real instructions.

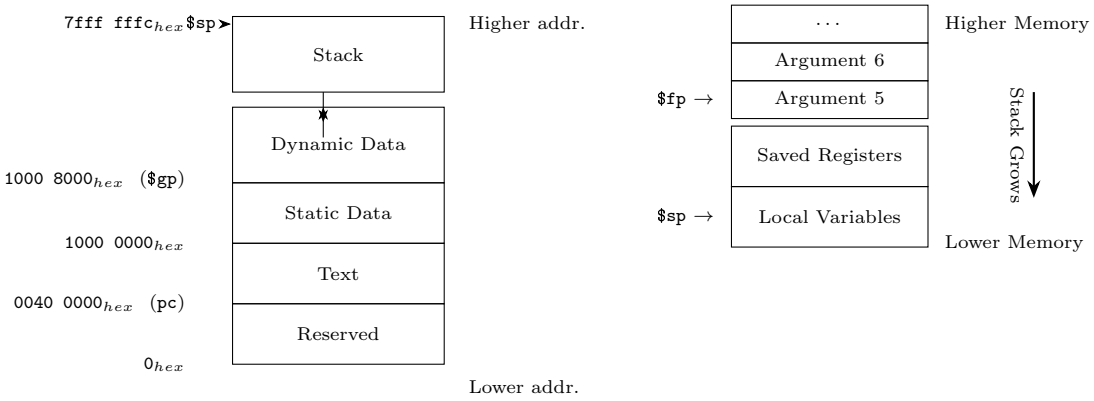
Name	Mnemonic	Operation
Branch Less Than	`blt`	`if(R[rs]<R[rt]) PC = Label`
Branch Greater Than	`bgt`	`if(R[rs]>R[rt]) PC = Label`
Branch Less Than or Equal	`ble`	`if(R[rs]<=R[rt]) PC = Label`
Branch Greater Than or Equal	`bge`	`if(R[rs]>=R[rt]) PC = Label`
Load Immediate	`li`	`R[rd] = immediate`
Move	`move`	`R[rd] = R[rs]`

6 Registers

6.1 Register name, number, use, call convention

Name	Number	Use	Preserved across a call?
`$zero`	0	The constant value 0	n/a
`$at`	1	Assembler temporary	No
`$v0--$v1`	2–3	Function results, expression eval	No
`$a0--$a3`	4–7	Arguments	No
`$t0--$t7`	8–15	Temporaries	No
`$s0--$s7`	16–23	Saved temporaries	Yes
`$t8--$t9`	24–25	Temporaries	No
`$k0--$k1`	26–27	Reserved for OS kernel	No
`$gp`	28	Global pointer	Yes
`$sp`	29	Stack pointer	Yes
`$fp`	30	Frame pointer	Yes
`$ra`	31	Return address	Yes

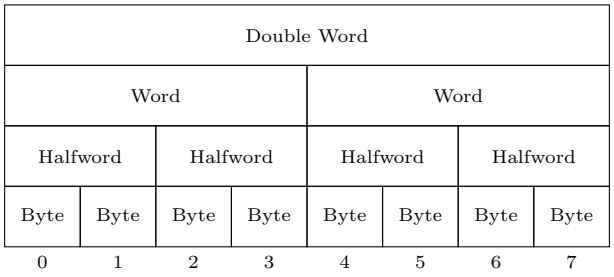
6.2 Memory map and stack frame



7 Data Sizes, Alignment & Number Formats

7.1 Data alignment

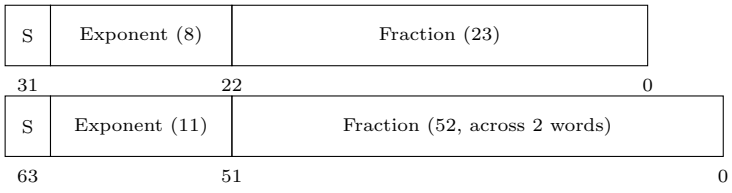
A word begins at an address divisible by 4; a halfword at an address divisible by 2. MIPS is **big-endian**.



7.2 IEEE 754 floating point

$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

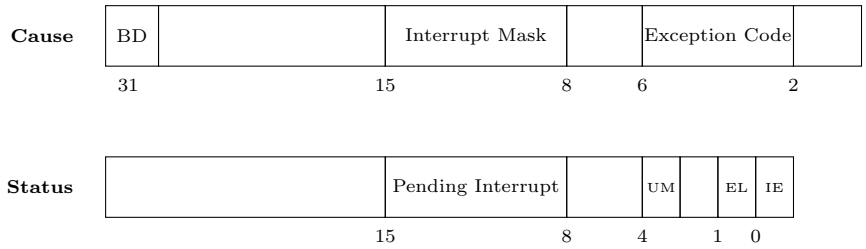
Single precision bias = 127; double precision bias = 1023; S.P. MAX exponent field = 255; D.P. MAX exponent field = 2047.



Exponent	Fraction	Object
0	0	±0
0	$\neq 0$	± Denormalized number
1 to MAX−1	anything	± Floating-point number
MAX	0	$\pm\infty$
MAX	$\neq 0$	NaN

8 Exception Handling

8.1 Cause and Status registers



BD = Branch Delay, UM = User Mode, EL = Exception Level, IE = Interrupt Enable.

8.2 Exception codes

#	Name	Cause
0	Int	Interrupt (hardware)
4	AdEL	Address Error Exception (load or instruction fetch)
5	AdES	Address Error Exception (store)
6	IBE	Bus Error on Instruction Fetch
7	DBE	Bus Error on Load or Store
8	Sys	Syscall Exception
9	Bp	Breakpoint Exception
10	RI	Reserved Instruction Exception
11	CpU	Coprocessor Unimplemented
12	Ov	Arithmetic Overflow Exception
13	Tr	Trap
15	FPE	Floating Point Exception

9 Opcode & Funct Field Quick Lookup

Every instruction table above already lists its `opcode/funct` in hex, which is what you need 95% of the time (looking up *one* mnemonic's encoding, or decoding one field you already isolated). The original green card additionally packs a full 128-row *binary → opcode/funct mnemonic → decimal → hex → ASCII* conversion grid (the card's own page 3, printed as the left column of *PDF page 2*) – a byte-value cross-reference used both for decoding raw instruction words *and* for looking up ASCII character codes. That grid is already about as compact as such a table can be; reformatting it here would not make it more readable, so it is intentionally omitted – refer to the original card (`mips-reference-data.pdf`, PDF page 2, left column, under "OPCODES, BASE CONVERSION, ASCII SYMBOLS") when you need it.

9.1 Base conversion notes

- opcode(31:26) == 0 ⇒ use the funct field (column 1 above) to find the R-format instruction.
- opcode(31:26) == 17_{10} (11_{hex}) ⇒ COP1 (floating point): if fmt(25:21) == 16_{10} (10_{hex}), f =`s` (single); if fmt(25:21) == 17_{10} (11_{hex}), f =`d` (double).

10 Size Prefixes

(10^x for disk/communication sizes; 2^x for memory sizes)

Prefix	10^x	2^x
Kilo-	10^3	2^{10}
Mega-	10^6	2^{20}
Giga-	10^9	2^{30}
Tera-	10^{12}	2^{40}
Peta-	10^{15}	2^{50}
Exa-	10^{18}	2^{60}
Zetta-	10^{21}	2^{70}
Yotta-	10^{24}	2^{80}

Prefix	Value
milli-	10^{-3}
micro-	10^{-6}
nano-	10^{-9}
pico-	10^{-12}
femto-	10^{-15}
atto-	10^{-18}
zepto-	10^{-21}
yocto-	10^{-24}

The symbol for each prefix is just its first letter, except μ is used for micro.

Source: Patterson & Hennessy, Computer Organization and Design*, 4th ed., © 2009 Elsevier Inc. Content reorganized for readability; all data reproduced from the original reference card.*